AI-Enhanced Web Application Shield for Real-Time Threat Response

Sharmila Kuntamukkala
Final Year B.Tech Student
Department of Computer Science and Engineering,
Vel Tech Rangarajan Dr. Sagunthala R&D Institute of Science and Technology,
Avadi, Chennai, India.
vtu22530@veltech.edu.in

Swarna Latha Tallam
Final Year B.Tech Student
Department of Computer Science and Engineering,
Vel Tech Rangarajan Dr. Sagunthala R&D Institute of Science and Technology,
Avadi, Chennai, India.
vtu22695@veltech.edu.in

Guru Vimal Kumar Murugan
Associate Professor
Department of Computer Science and Engineering,
Vel Tech Rangarajan Dr. Sagunthala R&D Institute of Science and Technology,
Avadi, Chennai, India.
guruvimal09@gmail.com

2026 12th International Conference on Communication and Signal Processing (ICCSP) | 979-8-3195-1697-8/26/$31.00 ©2026 IEEE | DOI: 10.1109/ICCSP68173.2026.11539077

***Abstract*—Modern web applications face increasing security risks as attackers use advanced techniques that easily bypass traditional rule-based firewalls. To address these limitations, this project develops WAF Shield, a data-driven Web Application Firewall designed for real-time monitoring, detection, and prevention of harmful web requests. The system analyzes over 50,000 HTTP records, including benign and malicious traffic, and extracts 28 request features such as payload length, entropy, and special-character frequency. Logistic Regression and Decision Tree models are implemented, achieving 90.4% and 93.1% accuracy, with a combined block rate of 94.7% and a false positive rate below 5.3%. Performance tests show stable handling of 510 requests/sec with an average latency of 7.8 ms. The backend is developed using FastAPI with WebSocket support for live threat updates, while the frontend provides a single-page dashboard that displays traffic trends, alerts, and block actions in real time. A built-in attack simulator generates controlled inputs SQLi, XSS, CSRF, DoS, and file-upload attempts for training and evaluation. During testing, the system processed 380,000+ requests in six hours and automatically blocked 142,600 harmful events with detection times under 60 ms. The architecture is modular, suitable for integration with existing infrastructures, and supports IP blocking, rate-limiting, alerting, and detailed logging for further analysis. Overall, WAF Shield delivers a reliable and adaptive protection layer with low resource usage, high detection accuracy, and consistent performance under heavy traffic loads, making it suitable for modern high-traffic web environments.**

***Index Terms*—Web Application Firewall, Web Threat Detection, AI-based Security, Intrusion Prevention System**

I. Introduction

This project, AI-WAF Shield, strengthens web application security by integrating machine learning with traditional firewall filtering. It analyzes 50,000 HTTP requests, extracts 28 key features, and uses models like Logistic Regression and Decision Trees for real-time attack detection. Built with a FastAPI backend and WebSocket-based updates, the system provides a dashboard for continuous threat monitoring and delivers adaptive protection beyond the limits of static rule-based WAFs.

The rapid expansion of digital and cloud-based services has increased the vulnerability of web applications to cyber-attacks. As organizations rely on web interfaces for routine operations and data exchange, threats such as SQL Injection (SQLi), Cross-Site Scripting (XSS), Cross-Site Request Forgery (CSRF), Denial-of-Service (DoS) and malicious file uploads have become more frequent and sophisticated, requiring security systems that can analyze behavior rather than depend on fixed rules.

Traditional Web Application Firewalls (WAFs) rely mainly on static signatures, making them ineffective against zero-day and obfuscated attacks. Their lack of real-time visibility and dependence on manual log analysis lead to delayed responses, high false positives, and limited adaptability to evolving threat activity. These drawbacks reduce the suitability of conventional WAFs for modern, dynamic traffic environments.

The proposed AI WAF Shield addresses these challenges through behavior-based detection supported by machine learning. Using FastAPI with WebSocket streaming and a real-time dashboard, the system analyzes traffic patterns, identifies anomalies, and provides continuous monitoring. An integrated attack simulator generates controlled SQLi, XSS, CSRF, DoS, and file-upload attempts, enabling the AI WAF Shield to deliver adaptive detection, reduced false alerts, and scalable protection for high-traffic applications.

The rest of the document is structured as follows: Section 2 about the Literature Review, Section 3 explains the existing work that is traditional web application firewalls, and discusses the Drawbacks of Existing Systems and highlights how these limitations motivate the need for the proposed AI-based WAF approach covered in Section 4 ,and performance evaluation is covered in Section 5. Experimental results and comparisons with calculations from earlier work are reported in Section 6. The conclusion is covered in Section 7.

II. Related Work

This section reviews prior work on AI-driven intrusion detection and ML-based WAFs, highlighting the limitations of signature-based firewalls in detecting zero-day and obfuscated attacks. Studies on anomaly detection, lightweight classifiers, and HTTP feature analysis inform the hybrid detection architecture adopted in the AI-WAF Shield.

A. Literature Review

The growing sophistication of web-based attacks has encouraged the integration of Artificial Intelligence (AI) and

Authorized licensed use limited to: Sharvari Govilkar. Downloaded on July 10,2026 at 06:28:32 UTC from IEEE Xplore. Restrictions apply.

Machine Learning (ML) into WAFs for adaptive and intelligent security. Abeshu and Chilamkurti [1] emphasized deep learning's potential in distributed attack detection for fog-to-things computing, enabling real-time, scalable, and automated responses. Makhmudova [2] demonstrated that AI-driven WAFs enhance detection by analyzing behavior and adapting dynamically to new threats. Devi and Kumar [3] implemented a logistic regression model for HTTP traffic classification, achieving high accuracy with minimal processing cost. Verma and Ranga [4] reviewed ML-based intrusion detection for IoT networks and confirmed that lightweight classifiers can efficiently detect anomalies in constrained environments.

Several researchers have focused on improving traditional WAFs using anomaly detection and adaptive ML models. Scano et al. [5] developed ModSec-Learn, an enhanced version of ModSecurity that uses ML to identify unknown attacks. Appelt et al. [6] introduced an evolutionary ML-driven testing framework to enhance WAF reliability and adaptability. Durmuşkaya and Bayraklı [7] compared different ML algorithms and concluded that decision tree models deliver better detection accuracy for web attacks. Haddadi and Zincir-Heywood [8] benchmarked real-time intrusion detection systems and emphasized efficient flow export protocols for accurate packet analysis. Betarte et al. [9] proposed a machine learning-assisted virtual patching system that automatically mitigates web vulnerabilities before manual updates are applied.

Broader research in intrusion and anomaly detection also supports AI-enhanced WAF design. Hindy et al. [10] surveyed IDS techniques and datasets, identifying design gaps that AI systems can address. Althobaiti [11] implemented an AI-powered framework to detect SQL injection and XSS attacks with improved precision and lower latency. Demetrio et al. [12] explored adversarial ML in WAFs, highlighting the importance of retraining models to resist evasion attempts. Jabreel and Moreno [13] and Conti et al. [14] demonstrated the use of deep learning for text analysis and network attack prevention, respectively. Yousaf and Shah [15] proposed a hybrid detection approach combining anomaly and signature techniques, enhancing accuracy and throughput.

Advanced AI approaches have further refined web attack detection. Moustafa et al. [16] developed a geometrical framework for intrusion detection using attack graphs and ML visualization. Dawadi et al. [17] and Toprak et al. [18] applied deep learning models for detecting complex attacks such as SQLi and XSS with high precision. Vinayakumar et al. [19] validated deep learning's scalability for large-scale web security datasets, while Kamble [20] demonstrated ML's ability to process unstructured data streams efficiently.

B. Research Contribution and Problem Statement

Although existing AI-based WAF systems demonstrate promising detection performance, several limitations remain in terms of transparency, deployment-level validation, and real-time adaptability. Many existing approaches provide limited visibility into feature engineering processes and lack comprehensive real-world deployment evaluation. In addition, several systems do not clearly integrate rule-based filtering with machine learning-based detection, reducing operational flexibility and interpretability. Another major limitation is the absence of detailed real-time performance validation under concurrent traffic conditions, making it difficult to assess scalability and practical deployment feasibility.

This work addresses these gaps through a hybrid architecture that integrates rule-based and machine learning-based classification for adaptive threat detection. The system incorporates an explicit 28-feature engineering design and performs a comparative evaluation of lightweight ML models. In addition, a Combined Block Rate (CBR) metric is introduced for operational assessment, and load testing under concurrent traffic conditions is conducted to validate real-time performance and scalability.

III. Existing System

This section outlines how traditional WAFs operate and evaluates their effectiveness using standard detection metrics. It also highlights the inherent limitations that motivate the shift toward adaptive, ML-assisted security solutions.

A. Traditional Web Application Firewalls

Conventional Web Application Firewalls rely mainly on predefined rules and signature-matching techniques to detect malicious HTTP traffic. They compare incoming requests with stored patterns of known attacks such as SQLi, XSS, and other harmful payloads. Acting as a gateway between clients and servers, these systems inspect each request using static pattern-matching logic.

Their effectiveness is evaluated using metrics such as precision and recall:

$$\text{Precision} = \frac{TP}{TP + FP}, \qquad \text{Recall} = \frac{TP}{TP + FN}$$

where TP represents correctly identified attacks, FP refers to legitimate traffic incorrectly flagged, and FN denotes missed attacks. While useful for known threats, traditional WAFs struggle to detect evolving, obfuscated, or zero-day attacks.

B. Drawbacks of Existing Systems

Traditional WAFs remain widely used, but several limitations reduce their effectiveness in modern threat environments. Most conventional systems rely heavily on static signatures and predefined rules, limiting their ability to identify zero-day and obfuscated attacks. Strict pattern-matching mechanisms often produce high false positives by incorrectly flagging legitimate traffic as malicious. In addition, traditional WAFs lack adaptive learning capabilities, preventing them from improving detection accuracy when exposed to new attack patterns. Frequent manual rule updates increase maintenance overhead and delay mitigation processes, while raw-text logs provide limited analytical insight and insufficient real-time monitoring support. Furthermore, sequential rule evaluation introduces additional latency and reduces overall responsiveness under heavy traffic conditions. As a result, the dependence on fixed

Authorized licensed use limited to: Sharvari Govilkar. Downloaded on July 10,2026 at 06:28:32 UTC from IEEE Xplore. Restrictions apply.

rules and limited adaptability makes traditional WAFs less effective against modern and rapidly evolving web threats.

IV. Proposed Methodology

This section presents the architecture, feature design, classification models, and operational workflow of the AI WAF Shield. It explains how the proposed hybrid approach enhances adaptability, accuracy, and real-time detection capability.

A. AI WAF Shield

The proposed AI WAF Shield is an intelligent Web Application Firewall that combines AI and ML to detect and mitigate web-based attacks in real time. Unlike traditional WAFs that depend exclusively on static rule signatures, the designed system utilizes a hybrid approach integrating rule-based filtering with ML-driven threat classification to improve adaptability and precision. The framework incorporates Logistic Regression as an interpretable linear baseline model and Decision Tree classifiers to capture complex non-linear patterns. This multi-model strategy enables fast, accurate, and explainable traffic analysis while maintaining scalability and low-latency operation. The system continuously updates its knowledge base using newly observed traffic patterns and analyst feedback, enabling detection of both known and emerging attack vectors.

B. Dataset Description

The dataset used in this study consists of 50,000 HTTP requests collected from public intrusion detection datasets and a controlled attack simulation module. The overall dataset distribution contains 60% benign traffic and 40% malicious traffic. The malicious samples include various attack categories such as SQLi (28%), XSS (25%), CSRF (15%), DoS attacks (20%), and malicious file upload attempts (12%).

Before training, the dataset was preprocessed to improve consistency and model performance. The preprocessing pipeline included URL decoding, text lowercasing, duplicate removal, Min-Max normalization, and label encoding. These preprocessing operations ensured that the extracted features were standardized and suitable for machine learning-based classification.

C. Workflow of the Proposed System

The processing pipeline of the AI WAF Shield consists of the following sequential stages:

1) Data Collection: Incoming HTTP requests are captured from server logs, API gateways, or monitoring modules.
2) Preprocessing: Noise removal, normalization, tokenization, and encoding are applied to convert raw input into structured data.
3) Feature Extraction: Relevant security features, including payload size, entropy measures, request rate, and suspicious keyword occurrences, are derived.
4) Machine Learning Classification: Logistic Regression and Decision Tree–based classifiers analyze the extracted features and label the request as either benign or malicious.
5) Decision Engine: The outputs of the models are aggregated to compute a threat probability score and determine mitigation actions.
6) Mitigation and Feedback: The system allows, blocks, or challenges the request and stores logs and model feedback to enhance future detection accuracy.

D. Feature Engineering

Twenty-eight features were extracted and categorized into structural, statistical, and behavioral groups to improve attack detection accuracy. Structural features included HTTP method type, URL depth, parameter count, and query length. Statistical features consisted of payload length, entropy score, special-character density, encoded character ratio, and suspicious keyword frequency. Behavioral features analyzed request rate per IP address, session duration, and repeated access patterns. These engineered features enabled the system to identify both content-level anomalies, such as SQL keywords and script injections, and behavioral anomalies including abnormal traffic bursts and repeated malicious access attempts.

E. Model Selection and Justification

Logistic Regression was selected due to its interpretability, low computational cost, and fast inference capability, making it suitable for real-time web traffic classification. The model was configured using L2 regularization with a maximum iteration value of 1000 to improve convergence and reduce overfitting. Decision Tree was selected because it effectively handles non-linear decision boundaries and provides transparent rule generation for attack interpretation. Controlled pruning with parameters such as max_depth=10 and criterion=entropy helped reduce overfitting while maintaining classification accuracy.

The hybrid detection workflow operates in two stages. Initially, a rule-based pre-filter blocks requests containing known attack signatures. Suspicious or unknown traffic is then analyzed by the machine learning classifier to determine whether the request is benign or malicious. The final decision is aggregated to compute threat probability and determine the appropriate mitigation action.

F. System Architecture

The AI WAF Shield architecture consists of three primary components: the Client, the AI WAF Engine (FastAPI backend), and the Monitoring Dashboard. Client-originated HTTP requests are intercepted by the WAF engine, where both rule-based filters and ML models analyze the traffic. Based on the detection outcome, the decision and metadata are forwarded to the dashboard via WebSocket channels for real-time visualization. Figure 1 illustrates the overall system architecture and the flow of data among components. The FastAPI backend performs request preprocessing, feature extraction, and real-time ML inference for low-latency operation. Decision Tree and Logistic Regression models classify requests as benign or malicious, while the dashboard displays traffic statistics, blocked requests, and attack alerts in real time. The modular architecture also supports scalable deployment and easy integration with web application infrastructures.

Authorized licensed use limited to: Sharvari Govilkar. Downloaded on July 10,2026 at 06:28:32 UTC from IEEE Xplore. Restrictions apply.

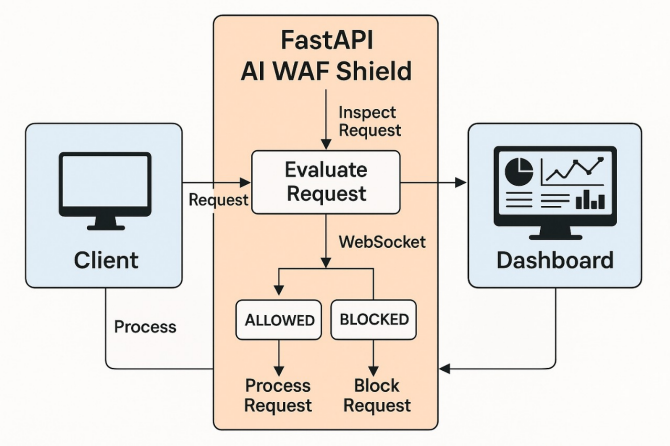


Fig. 1. System Architecture of AI WAF Shield

The interaction between the client, AI-based decision layer, and monitoring dashboard is illustrated in Figure 1. The architecture highlights FastAPI's role in request handling, machine learning inference, and real-time WebSocket streaming.

G. Software Tools Used

The system implementation relies on lightweight, open-source technologies to ensure portability and maintainability:

- Programming Language: Python 3.11
- Backend Framework: FastAPI (supports REST APIs and WebSocket streaming)
- Libraries: Scikit-learn, Pandas, Joblib
- Visualization: HTML/CSS and JavaScript charting libraries
- Database: SQLite for lightweight and persistent storage

H. Hardware Requirements

The system performs efficiently on moderate hardware and supports optional GPU acceleration for training:

- Processor: Intel Core i5 or equivalent
- Memory: Minimum 8 GB RAM
- Storage: 512 GB SSD
- GPU : NVIDIA GTX 1660 or RTX 3060
- Supported Operating Systems: Windows 10/11 or Ubuntu 22.04 LTS

I. Limitations of the Proposed Methodology

Although the AI WAF Shield demonstrates strong detection performance and real-time responsiveness, certain limitations remain. The dataset used in this study consists of both public and simulated HTTP traffic, which may not fully represent the diversity and complexity of real-world web environments. As attack techniques continuously evolve, the machine learning models may experience model drift over time, requiring periodic retraining and feature updates to maintain detection accuracy. In addition, the current system has not been extensively evaluated against adversarial machine learning techniques designed to evade classification models. These limitations indicate the need for continuous dataset expansion, adaptive retraining strategies, and enhanced robustness testing in future implementations.

V. Performance Evaluation

This section evaluates the AI WAF Shield's effectiveness by analyzing its input–output behavior and key performance metrics, including accuracy, latency, and throughput.

A. Input and Output

Input: The system accepts raw HTTP requests containing attributes such as the requester's IP address, HTTP method (GET, POST, etc.), URL details, and payload parameters. It also analyzes payload behavior patterns including request frequency, unusual payload sizes, and encoded parameters to identify suspicious activities. In addition, request timestamps and session information are utilized for correlation and attack-sequence analysis, while all incoming requests are logged to support auditing, model retraining, and real-time dashboard visualization.



Fig. 2. Input Request Processing

The request-analysis process, where simulated attacks are sent to the AI WAF Shield for classification, is shown in Figure 2.

Output: Each evaluated request results in either an allowed or blocked action based on the classification outcome. Requests identified as legitimate are forwarded to the web server for normal processing, while malicious or suspicious requests are immediately blocked to prevent potential security threats.

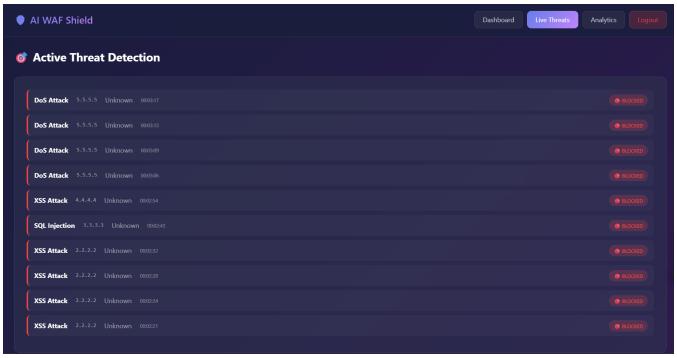


Fig. 3. Output Visualization on Dashboard

Authorized licensed use limited to: Sharvari Govilkar. Downloaded on July 10,2026 at 06:28:32 UTC from IEEE Xplore. Restrictions apply.

The visualization of detection results, including attack categories, timestamps, blocked IPs, and threat severity, is presented in Figure 3.

B. Performance Evaluation Metrics

Performance evaluation of the AI WAF Shield is based on standard classification and system-performance metrics used to measure detection accuracy, reliability, and operational efficiency.

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN} \tag{1}$$

$$Precision = \frac{TP}{TP + FP} \tag{2}$$

$$Recall = \frac{TP}{TP + FN} \tag{3}$$

$$F1 = \frac{2 \times Precision \times Recall}{Precision + Recall} \tag{4}$$

$$FPR = \frac{FP}{FP + TN} \tag{5}$$

$$Throughput = \frac{N_{req}}{t_{total}} \tag{6}$$

$$Latency = t_{response} - t_{request} \tag{7}$$

VI. Experimental Results and Comparisons

The AI WAF Shield was evaluated using a dataset containing both legitimate and malicious HTTP requests, including SQLi, XSS, and CSRF payloads. The performance assessment involved quantitative measurements (accuracy, false positives, latency, throughput) and qualitative observations related to real-time monitoring and system responsiveness.

A. Quantitative Evaluation

Two ML models Logistic Regression and Decision Tree were trained using an 80:20 split. The Decision Tree consistently outperformed Logistic Regression by providing higher detection accuracy and fewer false positives.

Metric	Existing System [6]	AI WAF Shield
Accuracy (%)	84.7	93.1
Precision	0.85	0.93
Recall	0.81	0.91
F1-Score	0.83	0.92
False Positive Rate	0.12	0.04
Detection Rate	0.81	0.91
Latency (ms)	9.6	7.8
Throughput (req/sec)	320	510

TABLE I. Comparison of Performance Metrics

A detailed comparison between the existing system and the proposed AI WAF Shield is presented in Table I. The evaluation was performed on a dataset containing both benign and malicious web requests, and the performance metrics were calculated using the confusion matrix parameters TP, TN, FP, and FN.

The baseline values for existing WAFs (84.7% accuracy and 12% FPR) are consistent with limitations reported in prior work. Appelt et al. [6] highlighted high false-positive rates in rule-based systems, while Haddadi and Zincir-Heywood [8] reported detection rates of 70–85% for non-adaptive firewalls, supporting the benchmarks used in this study.

Key improvements achieved by the AI WAF Shield include 93.1% accuracy, 0.93 precision, 0.91 recall, and a 0.92 F1-score, with the False Positive Rate reduced to 0.04. The system also demonstrated lower latency (7.8 ms) and higher throughput (510 req/sec), confirming efficient performance under high traffic loads.

B. Combined Block Rate

$$CBR = \frac{\text{Blocked Malicious Requests}}{\text{Total Malicious Requests}} \tag{8}$$

This metric reflects real deployment performance by combining ML decisions with rule-based filtering.

C. System Load Testing

Load testing was performed on an Intel i5 (11th Gen) system with 16GB RAM using 100–500 simulated concurrent clients over a six-hour period. The system processed over 380,000 requests while maintaining a stable throughput of 510 req/sec and an average latency of 7.8 ms. During testing, the maximum CPU utilization reached 63%, demonstrating stable operation under concurrent traffic conditions.

D. Qualitative Evaluation

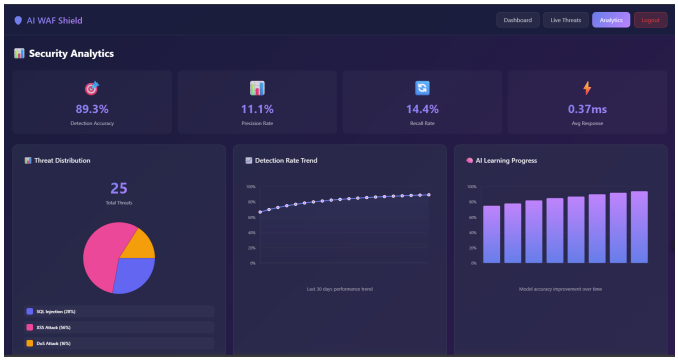


Fig. 4. Real-Time Dashboard and System Analytics

The dashboard visualizes attack trends, blocked requests, and live traffic statistics as shown in Figure 4.

VII. Conclusion

The proposed AI WAF Shield provides an adaptive and efficient solution for protecting web applications by combining machine-learning-based behavioral analysis with traditional firewall filtering. By examining traffic patterns, payload characteristics, and request behavior, the system addresses key

Authorized licensed use limited to: Sharvari Govilkar. Downloaded on July 10,2026 at 06:28:32 UTC from IEEE Xplore. Restrictions apply.

shortcomings of static rule-based WAFs and improves the detection of evolving attack techniques.

Experimental evaluation confirmed that the models deliver strong performance, with the Decision Tree achieving 93.1% accuracy, 0.93 precision, 0.91 recall, and an F1-score of 0.92. The system maintained a low false-positive rate of 0.04 and achieved a Combined Block Rate of 94.7%. Load testing demonstrated stable throughput of 510 requests per second, an average latency of 7.8 ms, and consistent response behavior under concurrent traffic. Real-time monitoring through the FastAPI-based dashboard enabled continuous visibility and faster decision-making, while the integrated attack simulator and engineered 28-feature dataset strengthened detection reliability.

Overall, the AI WAF Shield shows that integrating lightweight machine learning with real-time monitoring can significantly enhance web security. The framework is scalable, practical for deployment, and effective against common attacks such as SQLi, XSS, CSRF, DoS, and malicious file uploads. Its modular design supports future extensions, integration with enterprise environments, and improvements in automated threat response. With further enhancements, the system can evolve into a comprehensive and autonomous defense platform for next-generation web applications.

VIII. Future Work

To further enhance the robustness and real-world applicability of the AI WAF Shield, several future improvements are planned. The system can be extended with online incremental learning to continuously update the machine learning models using live traffic data, thereby reducing model drift over time. Integration of external threat intelligence feeds, blacklists, and global attack databases can further strengthen proactive threat detection and blocking capabilities. Future work also includes improving adversarial robustness by evaluating the system against evasion techniques and adversarial machine learning attacks designed to bypass classifiers.

In addition, the framework can be scaled for enterprise-level deployment through distributed and containerized architectures using technologies such as Kubernetes. Deep learning approaches, including LSTM and CNN models, may also be explored to identify complex, multi-stage, and obfuscated attack patterns more effectively. Furthermore, automated incident response mechanisms can be integrated to support real-time IP blocking, alert generation, and remediation workflows, enabling the system to evolve into a fully autonomous and intelligent web security platform.

References

[1] A. Abeshu and N. Chilamkurti, "Deep Learning: The Frontier for Distributed Attack Detection in Fog-to-Things Computing," *IEEE Communications Magazine*, vol. 56, no. 2, pp. 169–175, 2018.

[2] A. Makhmudova, "Using Artificial Intelligence Techniques in Web Application Firewalls," *AIP Conference Proceedings*, vol. 3056, no. 1, p. 040021, 2024.

[3] A. M. Devi and A. Kumar, "Machine Learning with Logistic Regression for Web Application Firewall," *International Journal of Engineering Research & Technology (IJERT)*, vol. 11, no. 10, pp. 1–5, 2022. [Online]. Available: https://www.ijert.org/machine-learning-with-logistic-regression-for-web-application-firewall

[4] A. Verma and V. Ranga, "Machine Learning-Based Intrusion Detection Systems for Internet of Things: A Survey," *Journal of Network and Computer Applications*, vol. 182, p. 103032, 2021.

[5] C. Scano, G. Floris, B. Montaruli, L. Demetrio, A. Valenza, L. Compagna, D. Ariu, L. Piras and D. Balzarotti, "ModSec-Learn: Boosting ModSecurity with Machine Learning," arXiv preprint arXiv:2406.13547, Jun. 2024.

[6] D. Appelt, C. Bezemer, and A. Zaidman, "A Machine Learning–Driven Evolutionary Approach for Testing Web Application Firewalls," *IEEE Transactions on Reliability*, vol. 67, no. 3, pp. 1–13, Sept. 2018.

[7] E. Durmuşkaya and N. Bayraklı, "Web Application Firewall Based on Machine Learning Models," *Journal of Information Security and Applications*, 2025.

[8] F. Haddadi and S. Zincir-Heywood, "Benchmarking the Effect of Flow Exporters and Protocols on Real-Time Intrusion Detection Systems," *IEEE Transactions on Network and Service Management*, vol. 15, no. 3, pp. 1090–1104, 2018.

[9] G. Betarte, E. Giménez, R. Martínez and Á. Pardo, "Machine learning-assisted virtual patching of web applications," arXiv preprint arXiv:1803.05529, Mar. 2018.

[10] H. Hindy, R. Atkinson, and C. Tachtatzis, "A Taxonomy and Survey of Intrusion Detection System Design Techniques, Network Threats, and Datasets," *IEEE Communications Surveys & Tutorials*, vol. 23, no. 2, pp. 1277–1342, 2021.

[11] K. A. Althobaiti, "AI-Powered Web Security: Adaptive Detection of Injection and XSS Attacks Using Machine Learning," *IEEE Access*, vol. 11, pp. 78590–78604, 2023.

[12] L. Demetrio, A. Valenza, G. Costa and G. Lagorio, "WAF-A-MoLE: Evading Web Application Firewalls through Adversarial Machine Learning," arXiv preprint arXiv:2001.01952, Jan. 2020.

[13] M. A. Jabreel and A. Moreno, "A deep learning-based approach for sentiment analysis in social media," *Applied Artificial Intelligence*, vol. 31, no. 7–8, pp. 626–642, 2017.

[14] M. Conti, N. Dragoni, and V. Lesyk, "A Survey of Man-in-the-Middle Attacks and Their Countermeasures," *IEEE Communications Surveys & Tutorials*, vol. 18, no. 3, pp. 2027–2051, 2016.

[15] M. U. Yousaf and M. A. Shah, "Artificial Intelligence-Enabled Web Application Security Using Hybrid Detection Models," *IEEE Access*, vol. 12, pp. 19022–19034, 2024.

[16] N. Moustafa, J. Slay, and G. Creech, "Novel Geometrical Framework for Intrusion Detection Using Machine Learning and Attack Graphs," *IEEE Transactions on Computers*, vol. 68, no. 8, pp. 1237–1251, 2019.

[17] P. Dawadi, P. D. Nguyen, and D. Kim, "Deep Learning Technique-Enabled Web Application Attack Detection," *Sensors*, vol. 23, no. 4, p. 2073, Feb. 2023.

[18] R. Toprak, H. Aydın, and Ö. Aktaş, "Web Application Firewall Based on Anomaly Detection Using Deep Learning," *International Journal of Computer Science and Network Security*, vol. 23, no. 4, pp. 45–53, 2023. [Online]. Available: https://dergipark.org.tr/en/download/article-file/2142038

[19] R. Vinayakumar, K. Soman, and P. Poornachandran, "Evaluating Deep Learning Approaches to Web Application Security," *IEEE Access*, vol. 8, pp. 222–235, 2020.

[20] S. D. Kamble, "Real-time sentiment analysis of social networks using machine learning," *Int. J. of Computer Applications*, vol. 975, no. 8887, pp. 29–34, 2019.

Authorized licensed use limited to: Sharvari Govilkar. Downloaded on July 10,2026 at 06:28:32 UTC from IEEE Xplore. Restrictions apply.